

```
In [1]: # Original graph
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['G'],
    'F': [],
    'G': []
}

FOUND = "FOUND"
CUTOFF = "CUTOFF"
FAILURE = "FAILURE"

def dls(node, goal, limit, path):
    # Goal found
    if node == goal:
        return FOUND, path

    # Depth Limit reached
    if limit == 0:
        return CUTOFF, None

    cutoff_occurred = False

    for child in graph[node]:

        # Path-Local cycle check
        if child in path:
            continue

        status, result = dls(
            child,
            goal,
            limit - 1,
            path + [child]
        )

        if status == FOUND:
            return FOUND, result

        if status == CUTOFF:
            cutoff_occurred = True

    if cutoff_occurred:
        return CUTOFF, None

    return FAILURE, None

def run_dls(limit):
    status, path = dls('A', 'G', limit, ['A'])
```

```

    return status, path

def iddfs(max_depth):
    limits_attempted = []

    for limit in range(max_depth + 1):
        limits_attempted.append(limit)

        status, path = dls('A', 'G', limit, ['A'])

        if status == FOUND:
            return status, path, limits_attempted

    return FAILURE, None, limits_attempted

# DLS tests
print("DLS RESULTS")
for limit in [1, 2, 3, 4]:
    status, path = run_dls(limit)
    print(f"Limit {limit}: {status}, Path: {path}")

# IDDFS tests
print("\nIDDFS RESULTS")
for maximum in [1, 2, 3, 4]:
    status, path, limits = iddfs(maximum)
    print(
        f"Maximum depth {maximum}: "
        f"Status = {status}, "
        f"Path = {path}, "
        f"Limits attempted = {limits}"
    )

```

DLS RESULTS

Limit 1: CUTOFF, Path: None
 Limit 2: CUTOFF, Path: None
 Limit 3: FOUND, Path: ['A', 'B', 'E', 'G']
 Limit 4: FOUND, Path: ['A', 'B', 'E', 'G']

IDDFS RESULTS

Maximum depth 1: Status = FAILURE, Path = None, Limits attempted = [0, 1]
 Maximum depth 2: Status = FAILURE, Path = None, Limits attempted = [0, 1, 2]
 Maximum depth 3: Status = FOUND, Path = ['A', 'B', 'E', 'G'], Limits attempted = [0, 1, 2, 3]
 Maximum depth 4: Status = FOUND, Path = ['A', 'B', 'E', 'G'], Limits attempted = [0, 1, 2, 3]

In []: